3D problem figures with `mechanicskit.scene3d`

From a scene spec to a textbook SVG: solids, hidden lines, shading, annotations and checks

Mirza Cenanovic

2026-10-13

Contents

1 What this is **1**

2 The CAD engine **3**

3 Installation **3**

4 Using it **4**
4.1 From the command line . . . 4
4.2 From Python . . . 4

5 The scene spec **4**
5.1 Parameters and expressions . . . 4
5.2 Parts . . . 5
5.3 Annotations . . . 5
5.4 Cameras, light and the final pick . . . 5
5.5 Checks . . . 6

6 How the stages work **6**
6.1 Shading in vector form . . . 6
6.2 Occlusion by ray casting . . . 6
6.3 Label checks . . . 7
6.4 The score . . . 7

7 The agent loop **7**

8 Extending it **7**

9 Performance and limits **8**

10 Files **8**

1 What this is

`mechanicskit.scene3d` draws a three-dimensional problem figure of the kind found in Meriam & Kraige or Hibbeler: a winch, a crank, a bracket, a shaft with its bearings. We describe the problem in a short JSON file, the *scene spec*, and the module returns an SVG in the book's figure standard. The 2D `sketch` module draws flat figures by hand-placing every line. This module builds the parts as solids and lets a CAD kernel work out which lines are visible from the chosen camera. A figure built from solids is dimensionally honest: the 250 mm on the drum is 250 mm in the model, the rope leaves the drum on a tangent because the model says so, and a change of camera is one number.

The pipeline has five stages, each a function in the module:

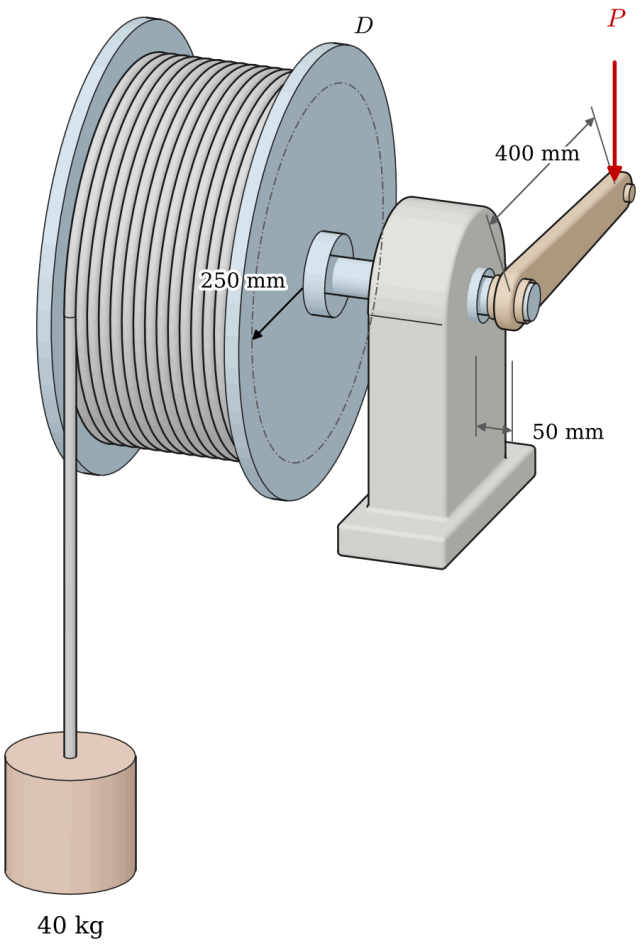


Figure 1: The Meriam & Kraige winch, drawn from `examples/scene3d/winch.json`. The dashed circle is the 250 mm pitch circle of the rope.

1. **Solids.** The parts are exact B-rep solids, built by named primitives from the problem's parameters.
2. **Hidden lines.** OpenCascade's hidden-line removal projects the solids and keeps the visible silhouettes and sharp edges as true vector curves.
3. **Shading.** The solids are tessellated, every triangle gets a Lambert shade band, a z-buffer finds the visible band at every pixel, and the regions are traced back into simplified vector fills.
4. **Annotations.** Forces, dimensions, radii, circles and labels are given in 3D and projected with the same camera, then drawn with `mechanicskit.sketch`, so they follow the house style.
5. **Checks and camera choice.** Every candidate camera is rendered and scored by ray-cast occlusion of the annotations and by label collisions. Physics checks run on the 3D model itself.

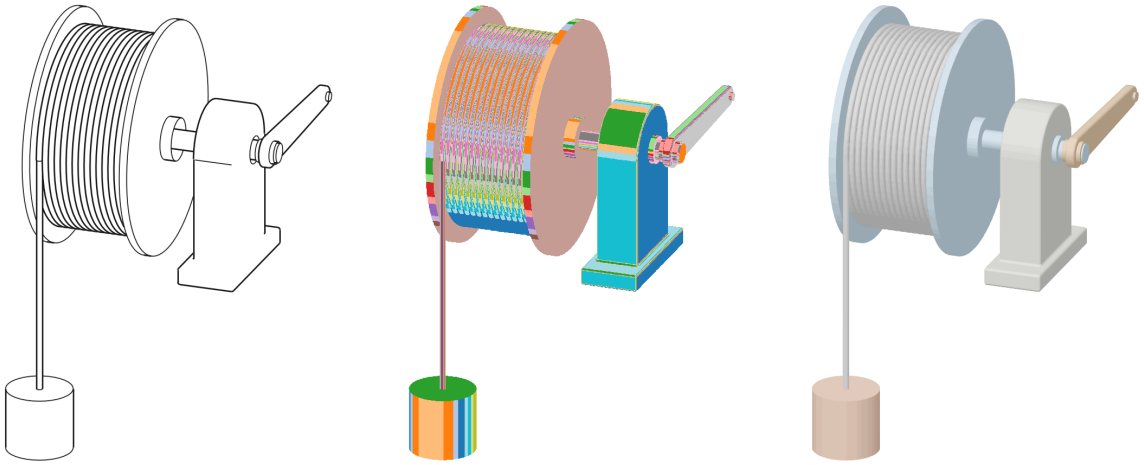


Figure 2: The three layers for one camera: the hidden-line edges, the z-buffer of shade groups (one colour per part and band), and the traced vector fills. The final figure stacks the edges on the fills and adds the annotations.

2 The CAD engine

The geometry kernel is **OpenCascade** (OCCT), the open-source B-rep kernel behind FreeCAD, CadQuery and KiCad's 3D viewer. We reach it through two Python layers:

- **build123d** (version 0.13 at the time of writing) is a Python CAD library on OpenCascade. It builds the solids: `Solid.make_cylinder`, `make_box`, `make_torus`, `make_hull`, `extrude`, `fuse` and `fillet`. Its sibling CadQuery uses the same kernel and would work equally well; build123d has the cleaner API and needs no conda.
- **OCP** is the set of Python bindings to OpenCascade that build123d installs (the `cadquery-ocp` wheel). We call OCP directly for three things that build123d does not expose in the form we need: the hidden-line algorithm `HLRBRep_Algo`, the curve sampler `GCPnts_QuasiUniformDeflection`, and the ray-shape intersector `IntCurvesFace_ShapeIntersector`.

The hidden-line removal is the exact algorithm, not the polygonal one. It works on the B-rep surfaces, so a cylinder's silhouette is a true line and an arc stays an arc. `HLRBRep_HLRToShape` returns the visible edges sorted by kind. We draw the *outline* edges (silhouettes, 0.9 pt) and the *sharp* edges (0.6 pt). We leave out the *smooth* edges, where two faces meet tangentially (fillet boundaries), as a draughtsman would.

Everything after the geometry is numpy and matplotlib: shading, the z-buffer, tracing and annotations. SVG output goes through matplotlib, so the figure inherits the book's fonts and `sketch` primitives.

3 Installation

The CAD dependency is optional, so the base `mechanicskit` install stays light. `import mechanicskit` does not import `scene3d`.

```
uv pip install -e "/home/mirza/python/MechanicsKit[cad]"      # a clone
uv pip install "mechanicskit[cad] @ git+https://github.com/cenmir/mechanicskit.git"
```

The `cad` extra pulls in `build123d>=0.9`, which brings OCP. Wheels exist for Python 3.10 to 3.13 on Linux, macOS and Windows; a dry run against the book's Python 3.13 venv resolves cleanly. The first import takes about two seconds while OpenCascade loads.

4 Using it

4.1 From the command line

```
python -m mechanicskit.scene3d examples/scene3d/winch.json
python -m mechanicskit.scene3d winch.json --out MachineElements/graphics/winch
```

This renders every camera in the spec into a folder named after the spec (or `--out`). The folder holds:

File	Content
`az20_el22.svg/.png`, …	one SVG and a 200 dpi PNG preview per candidate camera
`winch.svg`, `winch.png`	a copy of the chosen candidate, named after the spec's `name`
`report.json`	physics checks, per-camera checks and scores, the best and the used camera

The PNG previews are for looking at, by a person or by Claude. The SVG goes into the book.

4.2 From Python

```
from mechanicskit import scene3d as s3

report = s3.run("winch.json")                  # all cameras, best one copied out
report["best"]                                 # {'az': 20, 'el': 22}

# One camera by hand, e.g. in a notebook:
import json
spec = json.load(open("winch.json"))
P, parts = s3.build(spec)
rep = s3.render(spec, P, parts, s3.Camera(32, 26), "winch_az32")
```

`run` also accepts a dict, so a figure script can build the spec in Python and keep it next to the chapter's other figure code.

5 The scene spec

A spec is plain data. It holds no geometry code, which keeps the loop stable when an agent writes or edits it: a bad edit gives a bad picture or a clear error, never a half-built solid. Coordinates are in millimetres, with z up. The winch uses the shaft as the x axis.

5.1 Parameters and expressions

`params` holds the problem's numbers. Any value elsewhere in the spec may be a number or a string expression in these parameters (with `pi` and `sqrt` available), and a point is a list of such values:

```
"params": {"r_rope": 250, "d_rope": 16, "L_arm": 400, "gap": 50, "x_block": 480},
...
"p0": ["d_rope/2", "-r_rope", 0]
```

The dimensions of the problem statement are parameters, so the figure and the problem cannot drift apart.

5.2 Parts

Each part has an `id`, a primitive `kind`, a `color` (a palette name or any matplotlib colour) and `args`. The primitives shipped with the module are:

kind	Arguments	Use
`rod`	`p0, p1, r`	cylinder between two points: shafts, ropes, a hanging mass
`box`	`lo, hi`	axis-aligned box between two corners
`drum`	`x0, w, r_core, r_flange, t_flange, r_hub, hub_len`	flanged drum on the x axis, hub boss each side
`rope_wraps`	`x0, w, r, d`	rope wound on a drum, one torus per turn, `r` to the rope centre
`pillow_block`	`x0, x1, r_top, half_w, z_base, base_half_w, t_base`	bearing block, rounded housing on a base plate
`crank`	`x_mid, t, L, r_hub, r_tip, dir`	tapered crank arm in the plane $x =$ `x_mid`, with hub boss and tip pin

The palette names are `steel` (`sk.STEEL`, machine parts), `body` (`sk.BODY`, links and arms), `block`, `rope`, `mass` and `ground`. These follow the book's figure standard in `docs/figures.md`.

5.3 Annotations

Annotations are 3D objects. Their screen positions come from the same camera as the solids, so they stay attached to the geometry when the camera changes.

kind	Fields	Drawn as
`force`	`at, dir, length, text, head, role`	`sk.force`; `head: true` ends the arrow at `at` (a push). `role` is `load` (red, default), `resultant` (blue) or `internal` (green), and the label takes the arrow's colour
`dim`	`p0, p1, offset, dist, text, text_at`	extension lines along the 3D direction `offset`, a dimension line `dist` mm out, the value at its middle or `"outside"` past the end for short spans
`radius`	`centre, axis, r, angle_deg, r0, text, text_gap`	an arrow from radius `r0` to `r` in the plane normal to `axis`, at angle `angle_deg`
`circle`	`centre, axis, r`	a dash-dot circle, for a pitch circle or bolt circle
`label`	`at, text, role`	a haloed label at a 3D point

Every annotation accepts `screen_offset`, a shift in screen millimetres. This is how labels are moved clear of edges once the checks have said where they collide.

5.4 Cameras, light and the final pick

```
"cameras": [{"az": 20, "el": 22}, {"az": 32, "el": 26}, {"az": 45, "el": 30}],
"light": [-0.25, 0.45, 0.85],
"pick": {"az": 32, "el": 26}
```

The camera is orthographic, as in textbook axonometric drawings. Azimuth 0 looks along $+y$ with the viewer on the $-y$ side, a positive azimuth swings the viewer towards $+x$, and the elevation lifts it. The view direction and screen axes are

$$\mathbf{e} = \begin{bmatrix} \cos\epsilon \sin\alpha \\ -\cos\epsilon \cos\alpha \\ \sin\epsilon \end{bmatrix}, \qquad \mathbf{X} = \frac{\mathbf{e}_z \times \mathbf{e}}{\|\mathbf{e}_z \times \mathbf{e}\|}, \qquad \mathbf{Y} = \mathbf{e} \times \mathbf{X},$$

and a point $\mathbf{p}$ lands at $(\mathbf{p} \cdot \mathbf{X},\ \mathbf{p} \cdot \mathbf{Y})$ on the page. The HLR projector is built from the same frame (`gp_Ax2` with main direction $\mathbf{e}$ and x direction $\mathbf{X}$), so the edges, the fills and the annotations share one coordinate system with no fitting step.

`light` is given in the camera frame ($\mathbf{X}$ right, $\mathbf{Y}$ up, $\mathbf{e}$ towards the viewer), so the key light stays at the viewer's upper left whichever camera wins. An optional `pick` overrides the scored choice; without it, the lowest score is used.

5.5 Checks

```
"checks": {"rope_tangent": {"rope": "rope", "axis": [[0,0,0],[1,0,0]], "r": "r_rope"},
           "load_below":   {"load": "mass", "above": "drum"}}
```

`rope_tangent` measures the distance from the drum axis to the rope's top point against the given radius, and the cosine between the rope direction and that radius. Both are zero for a rope that leaves on a tangent. `load_below` reports the vertical clearance between the top of the load and the bottom of the drum. More checks of this kind (a belt tangent to two pulleys, a pin inside its hole) belong in `physics_checks`.

6 How the stages work

6.1 Shading in vector form

A textbook figure has flat or gently graded pastel fills, not photorealism, and we want them as vector paths. Per-triangle polygons in the SVG would give thousands of seams and a file of tens of megabytes. Sorting faces back to front (the painter's algorithm) fails where a long shaft passes through a bearing block. We therefore separate *visibility* from *output*:

1. Each part is tessellated (0.6 mm chord tolerance). Back-facing triangles are dropped. Each remaining triangle gets the Lambert intensity $I = 0.25 + 0.75\max(0, \mathbf{n} \cdot \ell)$ and is quantised into one of 14 bands. The pair (part, band) is its *group*.
2. A z-buffer 1800 pixels wide rasterises all triangles with barycentric depth interpolation and stores, per pixel, the group nearest the viewer. This raster decides visibility only and never appears in the output.
3. Each group's pixel mask is traced into closed contours (marching squares through `contourf`). The contours are simplified with Douglas–Peucker at 0.7 pixel and drawn as one `PathPatch` per group. A per-part underlay at mid tone sits beneath the bands, so a hairline between two bands never shows white.

A band of intensity I gets the colour $\mathbf{c}_{\text{dark}} + (\mathbf{c}_{\text{light}} - \mathbf{c}_{\text{dark}})I$, with $\mathbf{c}_{\text{dark}} = 0.7\,\mathbf{c}$ and $\mathbf{c}_{\text{light}} = \mathbf{c} + 0.45(\mathbf{1} - \mathbf{c})$ for the part's base colour $\mathbf{c}$. Fourteen bands read as a gradient on cylinders and as one flat tone on planes. The winch SVG is 0.8 MB with simplification and was 10 MB without it.

6.2 Occlusion by ray casting

Each annotation leaves 3D sample points along its lines. From every sample, a ray goes towards the viewer (direction $\mathbf{e}$) and `IntCurvesFace_ShapeIntersector` reports whether it hits a solid. The occluded fraction per annotation goes into the report. Samples within 3 mm of a solid are attachment points (the tip of P on the crank, an extension line starting on a face) and are skipped. This check caught the first version of the 250 mm radius arrow, which started inside the drum's hub boss.

6.3 Label checks

After drawing, matplotlib gives each label's bounding box in display coordinates. The report lists pairs of overlapping labels, labels whose box contains sampled points of the drawn edges (with the count), and labels that leave the figure.

6.4 The score

$$s = 10 \sum \text{occluded fractions} + 3\, n_{\text{overlaps}} + 0.02\, n_{\text{edge points under labels}} + 5\, n_{\text{labels outside}}$$

Occlusion dominates because a hidden dimension is wrong, while a label on an edge is only untidy. The weights are a starting point. For the winch, the three cameras scored 2.04, 2.22 and 2.30, and the remaining score at az 20° comes from attachment samples on P and the 400 mm dimension.

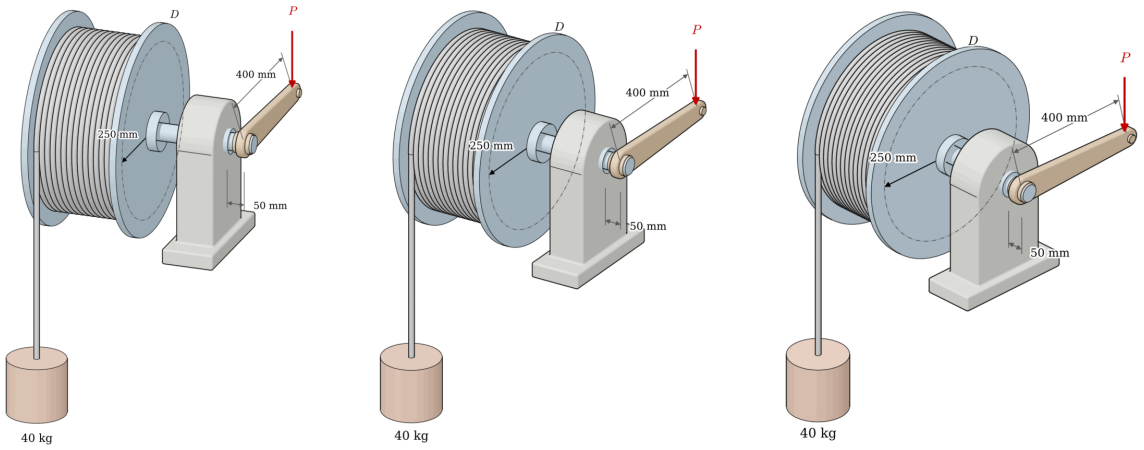


Figure 3: The three candidate cameras of the winch spec, az 20°/el 22°, az 32°/el 26° and az 45°/el 30°. All three pass the occlusion check; they differ in where the labels sit against the edges.

7 The agent loop

The module is built for Claude Code to drive, with the person reviewing. The loop that produced the winch figure was:

1. **Read the source figure** and write a spec: parameters from the problem statement, parts from the primitive table, annotations as in the original.
2. **Run** `python -m mechanicskit.scene3d spec.json` and **read** `report.json`.
3. **Look at the PNG previews** and critique them against a checklist: every dimension unobstructed and unambiguous; the rope hanging on a tangent and the load below the drum; arrows readable and not running along extension lines; labels clear of edges; the same view as the source unless there is a reason to change it.
4. **Edit the spec, not the code**: move an annotation's `angle_deg` or `screen_offset`, tilt a dimension's `offset`, add a `circle`, change a camera. Then run again.

The programmatic checks catch what a visual review misses, and the visual review catches what no check encodes. On the winch, the checks found the radius arrow inside the hub boss and the labels touching the mass outline. The visual review found the painter's-algorithm slivers at the bearing (fixed by the z-buffer), speckles between shade bands (fixed by dropping a blur), the extension line of 400 mm running along the P arrow, and a dark drum face (fixed by the camera-frame light). Each round takes about a minute for three cameras.

8 Extending it

A new part is a function that returns a build123d solid or a list of them, registered with the decorator. It is then available to every spec by its function name:

```python
import build123d as b
from mechanicskit import scene3d as s3

@s3.primitive
def pulley(x0, w, r, groove_r):
    """V-less flat pulley on the x axis with a rope groove."""
    body = b.Solid.make_cylinder(r, w, b.Plane(origin=(x0, 0, 0), z_dir=(1, 0, 0)))
    groove = b.Solid.make_torus(r, groove_r, b.Plane(origin=(x0 + w/2, 0, 0), z_dir=(1, 0, 0)))
    return body.cut(groove)

s3.run("hoist.json")        # parts may now use "kind": "pulley"
```

Keep primitives general (a drum, a bracket, a gear blank), with the problem's numbers in the spec. A new annotation kind goes into `annotate`, and it must return sample points so the occlusion check covers it. A new physics check goes into `physics_checks`.

9 Performance and limits

On this machine, building the winch takes about 3 s. Each camera takes about 20 s: HLR 2 s, the z-buffer about 12 s (a Python loop over some 60 000 triangles), tracing 3 s, ray casting 1 s. The z-buffer is the place to optimise if it matters, by vectorising over triangles of equal pixel size or by moving it to numba.

Known limits:

- No perspective. Textbook figures do not need it, and the HLR and the annotation projection would both need the focal length.
- Rope wraps are separate tori, not a helix. They look right at figure scale, and a helical sweep is possible but slows HLR considerably.
- Labels use `screen_offset` in screen millimetres, so a large camera change can need the offsets touched again. The label checks say which.
- Hidden edges are computed by HLR but not drawn. Dashed hidden lines for a technical drawing would be a small addition (`HCompound()`).
- The 50 mm dimension of the winch is drawn as the axial gap between the bearing face and the crank's mid-plane. The original figure is ambiguous on this point, and only the 250 mm and 400 mm enter the moment balance $P \cdot 400 = mg \cdot 250$.

10 Files

Path	Content
`mechanicskit/scene3d.py`	the module: primitives, camera, HLR, shading, annotations, checks, `run`, CLI
`examples/scene3d/winch.json`	the winch spec
`examples/scene3d/layers.py`	renders the layer figure of this document
`docs/scene3d.qmd`	this document; `docs/scene3d/` holds its images
`docs/figures.md`	the 2D figure standard that the annotations follow